

CS-405: Deep Learning

Week 7 - Long Short-Term Memory (LSTM)

Dr. Sajjad Hussain

SEECS, NUST

March 2nd, 2026

1 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM)

Why LSTM?

While GRU is powerful, LSTM was the **original** solution to the vanishing gradient problem (Hochreiter & Schmidhuber, 1997).

Key Innovations:

- **Cell State** (c^t) - Long-term memory highway
- **Input Gate** (i^t) - Controls what new info to add
- **Forget Gate** (f^t) - Controls what to remove
- **Output Gate** (o^t) - Controls what to output

Key Difference from GRU: LSTM has **separate memory** (cell state) and **hidden state**, while GRU combines them.

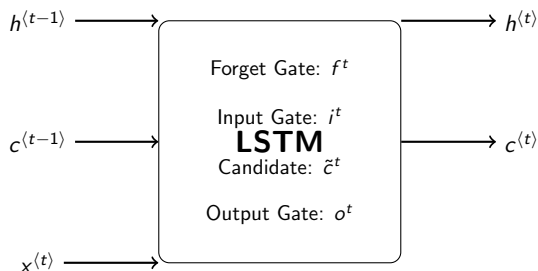
LSTM Variables

Symbol	Description
$x^{(t)}$	Current input
$h^{(t-1)}$	Previous hidden state (short-term memory)
$c^{(t-1)}$	Previous cell state (long-term memory)
$h^{(t)}$	New hidden state (output)
$c^{(t)}$	New cell state (updated long-term memory)
$f^{(t)}$	Forget gate (0 to 1) - what to forget from cell state
$i^{(t)}$	Input gate (0 to 1) - what new info to add
$o^{(t)}$	Output gate (0 to 1) - what to output from cell state
$\tilde{c}^{(t)}$	Candidate cell state (proposed new memory)

Dimensions:

- All gate vectors have size $n_h \times 1$ (same as hidden state)
- Cell state also has size $n_h \times 1$

LSTM Architecture - High Level



Key Insight: The cell state c^t runs straight through with only linear operations (add/multiply by gates) - this is the "gradient highway"!

LSTM Gates - Step by Step

All gates use the same inputs: $h^{\langle t-1 \rangle}$ and $x^{\langle t \rangle}$

- ① **Forget Gate:** What to discard from long-term memory

$$f^{\langle t \rangle} = \sigma(W_f \cdot [h^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_f)$$

- ② **Input Gate:** What new info to add

$$i^{\langle t \rangle} = \sigma(W_i \cdot [h^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_i)$$

- ③ **Candidate Cell State:** Proposed new memory

$$\tilde{c}^{\langle t \rangle} = \tanh(W_c \cdot [h^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_c)$$

- ④ **Output Gate:** What to output from cell state

$$o^{\langle t \rangle} = \sigma(W_o \cdot [h^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_o)$$

Note: All gates use sigmoid (σ) for values between 0 and 1. Only candidate uses tanh for values between -1 and 1.

LSTM Cell State Update - The Memory Highway

Cell State Update

$$c^{(t)} = \underbrace{f^{(t)} \odot c^{(t-1)}}_{\text{Forget part}} + \underbrace{i^{(t)} \odot \tilde{c}^{(t)}}_{\text{New part}}$$

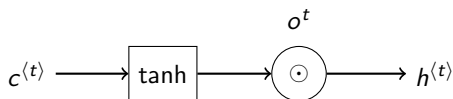
Interpretation:

- $f^t \approx 1, i^t \approx 0$: Keep old memory, ignore new
- $f^t \approx 0, i^t \approx 1$: Replace with new memory
- $f^t \approx 0.5, i^t \approx 0.5$: Mix old and new equally

LSTM Hidden State Output

Hidden State Update

$$h^{\langle t \rangle} = o^{\langle t \rangle} \odot \tanh(c^{\langle t \rangle})$$



What happens:

- 1 Cell state c^t is squashed to $[-1, 1]$ via $\tanh$
- 2 Output gate o^t decides how much of this to reveal
- 3 If $o^t \approx 0$: Hidden state is near zero (no output)
- 4 If $o^t \approx 1$: Hidden state reveals the cell state

Purpose: The network can hide information in cell state without revealing it in output!

Complete LSTM Cell - All Together

Three Stages:

- 1 **Forget:** $f^t \odot c^{t-1}$ (what to remove)
- 2 **Input:** $i^t \odot \tilde{c}^t$ (what to add)
- 3 **Output:** $o^t \odot \tanh(c^t)$ (what to reveal)

LSTM vs GRU - Key Differences

Feature	LSTM	GRU
Cell State (separate memory)	Yes (c^t)	No (combined)
Number of Gates	3 (f^t, i^t, o^t)	2 (z^t, r^t)
Forget Gate	Explicit	No (update gate combines)
Output Gate	Controls output	No (full hidden state output)
Parameters	More ($4n_h^2 + \dots$)	Fewer ($3n_h^2 + \dots$)
Training Speed	Slower	Faster
Performance on very long sequences	Slightly better	Slightly worse

When to use:

- **LSTM:** Very long sequences, need fine-grained control, more data available
- **GRU:** Shorter sequences, want faster training, less data, simpler model

LSTM Gradient Flow - The Highway

In RNN:

$$h^t = \tanh(W_{hh}h^{t-1} + \dots)$$

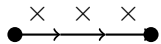
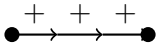
$$\frac{\partial h^t}{\partial h^{t-1}} = \text{diag}(1 - (h^t)^2) \cdot W_{hh}^T$$

In LSTM (for cell state):

$$c^t = f^t \odot c^{t-1} + i^t \odot \tilde{c}^t$$

Gradient through cell state:

$$\frac{\partial c^t}{\partial c^{t-1}} = \text{diag}(f^t) + \text{terms from gates}$$

RNN Gradient Path:  **LSTM Gradient Path:** 

Key Insight: If $f^t \approx 1$, then $\frac{\partial c^t}{\partial c^{t-1}} \approx 1$ (identity) - gradients flow through unchanged!

LSTM Example - The “France” Problem Revisited

Sentence

“I grew up in **France**. I moved to Germany for work. After 10 years, I now speak fluent **French**.”

At $t=5$ (seeing “France”):

- Forget gate $f^5 \approx 0.95$ for country dimension in cell state
- Input gate $i^5 \approx 0.3$ to add new info
- Cell state c^5 now strongly encodes “France” in long-term memory

At $t=6-18$ (filler words):

- Forget gate $f^t \approx 0.95$ for country dimension
- Cell state preserved: $c^t \approx 0.95 \times c^{t-1}$
- After 14 steps: $0.95^{14} \approx 0.49$ (still strong!)

At $t=19$ (seeing “French”):

- Input gate opens ($i^{19} \approx 0.4$) to combine “French” with stored “France”
- Output gate $o^{19} \approx 0.9$ reveals the combined representation

Numerical Comparison - RNN vs GRU vs LSTM

Tracking a single important dimension across 50 steps:

Steps	RNN	GRU	LSTM
	($\times 0.8$ each)	($z=0.95$)	($f=0.95$)
0	1.00	1.00	1.00
10	0.11	0.60	0.60
20	0.01	0.36	0.36
30	0.001	0.21	0.21
40	0.0001	0.13	0.13
50	0.00001	0.08	0.08

Key observation: GRU and LSTM have similar preservation power when gates are set appropriately!

But LSTM has advantages:

- Separate cell state means information can be **hidden** (output gate closed)
- Explicit forget and input gates give **finer control**
- Can handle **even longer** dependencies with proper gate values

LSTM Summary - All Equations in One Place

Forget Gate

$$f^{(t)} = \sigma(W_f \cdot [h^{(t-1)}, x^{(t)}] + b_f)$$

Input Gate

$$i^{(t)} = \sigma(W_i \cdot [h^{(t-1)}, x^{(t)}] + b_i)$$

Candidate Cell State

$$\tilde{c}^{(t)} = \tanh(W_c \cdot [h^{(t-1)}, x^{(t)}] + b_c)$$

LSTM Summary - All Equations in One Place

Output Gate

$$o^{(t)} = \sigma(W_o \cdot [h^{(t-1)}, x^{(t)}] + b_o)$$

Cell State Update (Long-term Memory)

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

Hidden State Output (Short-term Memory)

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

LSTM vs GRU - Practical Guidelines

Choose LSTM when:

- Sequences are **very long** (100+ steps)
- You need **fine-grained control** over memory
- You want to **hide** information (output gate)
- You have **plenty of data** and compute
- Working with **speech/audio** (often benefits from LSTM)

Choose GRU when:

- Sequences are **moderate length**
- You want **faster training**
- You have **limited data** (fewer parameters)
- You need a **simpler** model
- Working with **text** (GRU often sufficient)

Rule of Thumb: Start with GRU for quick prototyping, switch to LSTM if you need more capacity or see vanishing gradient issues.

Complete Architecture Comparison

Feature	RNN	GRU	LSTM
Hidden State	h^t	h^t	h^t
Cell State	No	No	c^t
Update Gate	No	z^t	No
Reset Gate	No	r^t	No
Forget Gate	No	No	f^t
Input Gate	No	No	i^t
Output Gate	No	No	o^t
Gradient Path	Multiplicative	Additive	Additive (cell)
Vanishing Gradients	Severe	Mild	Mildest
Parameter Count	Lowest	Medium	Highest
Training Speed	Fastest	Fast	Slowest
Long-range Memory	Poor	Good	Excellent

LSTM - Key Takeaways

- **LSTM** was designed to solve the vanishing gradient problem with a **cell state** highway
- **Three Gates:**
 - Forget gate (f^t): What to remove from long-term memory
 - Input gate (i^t): What new info to add
 - Output gate (o^t): What to reveal from memory
- **Cell State Update:**

$$c^t = f^t \odot c^{t-1} + i^t \odot \tilde{c}^t$$

- **Hidden State:**

$$h^t = o^t \odot \tanh(c^t)$$

- **Why it works:** If $f^t \approx 1$, gradients flow through cell state unchanged
- **LSTM vs GRU:**
 - LSTM: More gates, separate cell state, finer control, more parameters
 - GRU: Simpler, faster, fewer parameters, often sufficient
- **When to use:** LSTM for very long sequences and when you need maximum capacity; GRU for most other cases